EXP-Bench: Can AI Conduct AI Research Experiments?

Anonymous Author

Abstract

Most benchmarks assess isolated capabilities of AI systems, such as code generation or analysis, but do not evaluate the end-to-end rigor required to run complete scientific experiments. We introduce EXP-Bench, a benchmark that evaluates whether agents can design, implement, execute, and interpret AI research experiments grounded in peer-reviewed publications and their open-source code. EXP-Bench comprises 461 tasks extracted from 51 NeurIPS/ICLR 2024 papers (12,737 fine-grained subtasks) spanning CV, NLP, RL, and applications. Each task pairs a research question with a masked repository and execution-validated ground truth, and is scored by a multi-metric protocol: Design (D), Implementation (I), Execution (E), and Conclusion (C). D/I/C are judged by an LLM while execution is validated in controlled Docker environments with integrity monitoring. Across seven agent–model pairs, agents achieve partial correctness but rarely complete experiments end-to-end; conjunctive scoring (requiring D, I, C, and E) reduces apparent performance from double digits to near zero. EXP-Bench provides a high-fidelity, execution-grounded testbed revealing that current agents handle parts of experimentation yet struggle with complete, reliable research workflows.

AI-Generated

1 Introduction

Rigorous evaluation of AI systems has historically progressed through benchmarks that target specific skills: natural language understanding, knowledge recall, or coding. While such evaluations are essential, modern research increasingly relies on agents that must conduct multi-step scientific experiments: formulating hypotheses, selecting variables, writing and running code in specific environments, and interpreting outcomes. Existing benchmarks rarely assess this end-to-end rigor, leaving a gap between point-skill competence and the demands of actual AI research workflows.

We address this gap with **EXP-Bench**, a benchmark for end-to-end AI research experimentation. Tasks are anchored in real, peer-reviewed AI papers and their public code, pairing realistic research questions with masked repositories and execution-validated ground truths. EXP-Bench evaluates four linked phases—*Design* (D), *Implementation* (I), *Execution* (E), and *Conclusion* (C)—to measure whether an agent can complete coherent, reproducible experiments rather than isolated subtasks.

Design principles for EXP-Bench emphasize: (1) *realism* via tasks extracted from recent NeurIPS/ICLR papers with public code; (2) *coverage and diversity* across CV, NLP, RL, and applications; (3) *conjunctive validity* through phase-wise metrics and joint checks that reduce spurious success; (4) *reproducibility* via Dockerized execution with strict resource/time budgets; and (5) *integrity* through repository masking and behavior monitoring.

Our main findings show that contemporary agents exhibit partial competence but rarely complete experiments end-to-end. Conjunctive scoring greatly reduces apparent success, clarifying where systems fail—often in missing implementation components, environment setup, or drawing correct conclusions from runs. An overview of EXP-Bench appears in Figure 1; we summarize aggregate results in Table 1. Our contributions are:

Preprint. Under review.

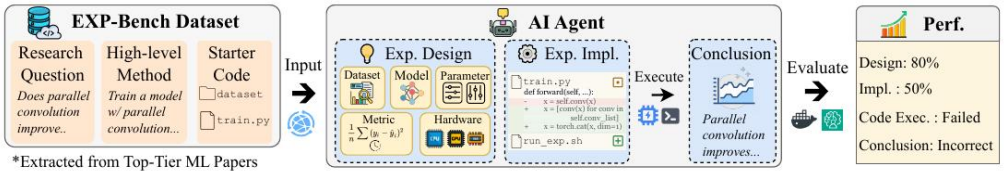

Figure 1: EXP-Bench evaluates whether agents can design, implement, execute, and conclude AI research experiments from peer-reviewed papers and code.

- **A benchmark for executable research experiments:** 461 tasks (12,737 subtasks) grounded in peer-reviewed papers with masked repos and validated ground truths.
- **A semi-automated construction pipeline:** multi-pass extraction from papers and code, followed by execution-based verification and human checks.
- **A rigorous evaluation protocol:** phase-wise D/I/E/C metrics with joint correctness checks, Docker-based execution validation, and integrity monitoring.
- **Empirical insights:** analysis across agents, models, and categories revealing systematic failure modes and the diagnostic value of conjunctive scoring.

2 Related Work

General-purpose and knowledge benchmarks. Large-scale evaluation suites such as BIG-bench [2] and MMLU [3] assess broad capabilities but do not require orchestrating complete experiments.

Agent and software engineering benchmarks. SWE-bench [4] measures repository-level issue resolution; it demands multi-file editing and build/test integration, a step toward realistic workflows but not explicitly focused on scientific experimentation. AgentBench [6] evaluates LLMs across interactive environments, highlighting decision-making and instruction following. OpenHands [7] provides a platform and evaluations for software-developer agents. These lines of work underscore the need for realistic, end-to-end assessments but stop short of measuring design-to-conclusion alignment for scientific experiments.

Scientific-agent evaluations. ScienceAgentBench [1] curates tasks from scientific workflows and uses execution signals to assess code-producing agents. EXP-Bench complements this direction by anchoring tasks in AI research papers and requiring a conjunctive alignment between design, implementation, execution, and conclusion.

3 Benchmark Design and Dataset

3.1 Target Capability and Task Definition

EXP-Bench is designed to evaluate whether an agent can conduct a coherent, end-to-end AI research experiment. Each task specifies a research question, a high-level method, datasets, metrics, and constraints derived from a target paper and its code repository. Inputs include a masked repository (to prevent shortcutting via answer-revealing files) and a task specification; the expected output comprises (i) a design plan, (ii) an implementation in the form of runnable edits or scripts, (iii) executable runs under a controlled environment, and (iv) a short conclusion interpreting results.

3.2 Dataset Scope and Composition

EXP-Bench contains 461 tasks sourced from 51 NeurIPS/ICLR 2024 papers spanning CV, NLP, RL, and applications, totaling 12,737 fine-grained subtasks. Category distribution indicates broad coverage (Figure 2), while an example task illustrates the level of detail provided to agents (Figure 8).

AI-Generated

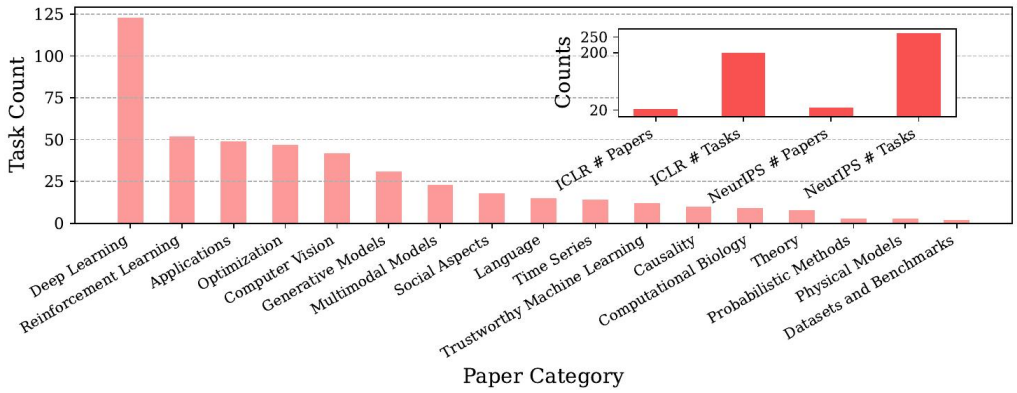

Figure 2: Category distribution across evaluated tasks in EXP-Bench.

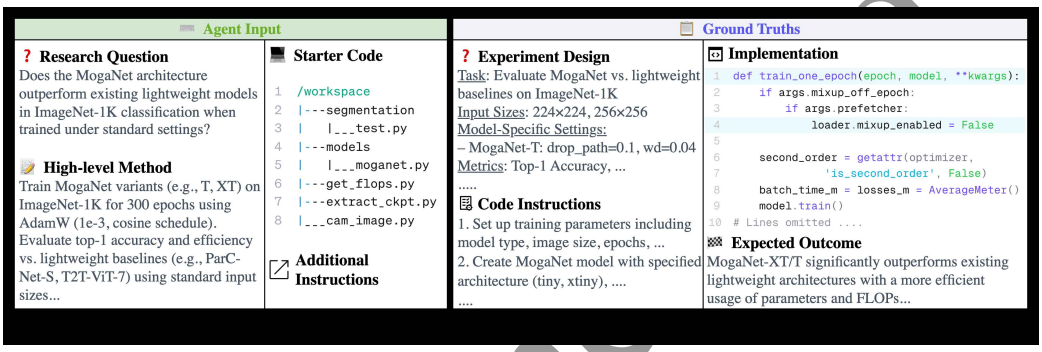

Figure 3: An example task extracted from ICLR 2024 MogaNet [5].

3.3 Construction Pipeline

Stage 1: Source selection. We identify papers with public repositories using citation and repository activity signals (e.g., stars/forks).

Stage 2: Experiment procedure extraction. (2.1) A multimodal extraction process parses PDFs, figures, and tables; retrieval-augmented, multi-pass prompts assemble research questions, methods, datasets, metrics, and constraints; cross-document re-querying recovers missing details. (2.2) A tool-augmented agent (PDF, shell, web) locates scripts, dependencies, and checkpoints; it outputs runnable script chains and usage instructions. AST tracing converts scripts into step-by-step implementation requirements.

Stage 3: Verification and refinement. Candidate scripts are executed in clean containers; outputs are compared to paper/code expectations and iterated upon. Lightweight human review cross-checks structured content, and answer-revealing files (e.g., READMEs or turnkey scripts) are masked via scripted git operations to prevent shortcutting. Quality control relies on execution-based validation and final human checks; we do not report inter-annotator agreement.

3.4 Evaluation Protocol and Metrics

The evaluation decomposes experiment success into four linked phases: Design (D), Implementation (I), Execution (E), and Conclusion (C). Design requires selecting variables, ablations, and configurations aligned with the task; Implementation requires producing runnable changes or scripts matching the design; Execution requires successful runs within constraints; and Conclusion requires interpreting results consistent with observed outputs.

Scoring uses multiple signals: D/I/C are judged by an LLM (o3-mini) using rubric-based prompts; E is validated by running agent-produced code in Docker (Ubuntu 24.04, up to 4 × A40 GPUs, 40-minute timeout). Conjunctive checks capture end-to-end correctness: I•E requires both implementation and

AI-Generated

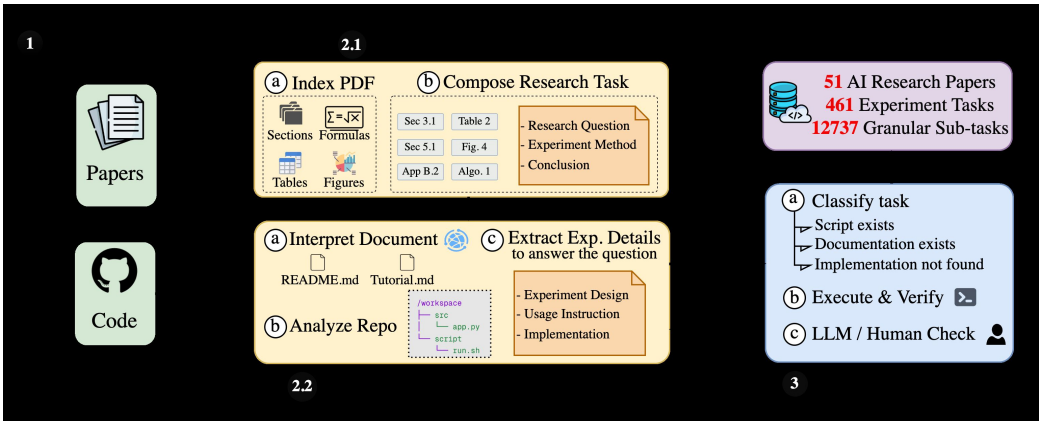

Figure 4: Semi-automated pipeline for constructing EXP-Bench from peer-reviewed papers and public code.

execution success; All✓ requires D, I, and C; and All•E✓ requires D, I, C, and E jointly. A behavior monitor flags disallowed behaviors such as accessing the paper, checking out different git commits, or generating fake data.

4 Benchmark Validation

We validate EXP-Bench along fidelity, reliability, and diagnostic value.

Task validity and realism. Tasks are anchored in peer-reviewed papers and their public code. Masking prevents trivial lookup, and AST-derived requirements preserve operational detail.

Execution-grounded quality control. Candidate implementations are executed in clean containers; mismatches trigger refinement. Final ground truths include runnable script chains and validated conclusions.

Leakage and integrity controls. Repositories are masked to remove answer-revealing artifacts. The monitor rejects policy-violating behaviors (e.g., external paper access).

Metric robustness and stability. We study sensitivity to scoring choices and noise (Figure 5 and Figure 6). Stricter metrics reduce overestimation by requiring agreement across phases, and we observe qualitative stability trends across agents and categories.

Scoring protocol and adjudication. Phases D, I, and C are judged against concise rubrics: design requires specifying variables, controls, and success criteria; implementation requires making the minimal, correct code or config changes relative to the masked repo; conclusions must logically follow from the observed run outputs. Execution (E) is validated by containerized runs with checks on exit codes, expected artifacts, and metric values within tolerance bands. Joint metrics (e.g., All•E✓) require that the same coherent attempt satisfies the constituent phases to avoid credit from inconsistent partials. Ambiguous or borderline cases include justification strings and are sampled into a 200-item calibration set for human review. The protocol favors precision over recall to limit optimistic inflation from generous judgments while retaining diagnostic granularity across phases.

5 Experimental Setup

We evaluate representative agents and models on EXP-Bench using the same resource and time budgets as in ground-truth validation: Dockerized Ubuntu 24.04 environments with up to 4 × A40 GPUs and a 40-minute per-task timeout. The monitor enforces integrity constraints. We consider seven agent–model pairs, including combinations of OpenHands (OH) and IterativeAgent (IA) with o3-mini, Claude-3.7 Sonnet, Claude-3.5 Haiku, DeepSeek R1, and Amazon Nova Pro.

AI-Generated

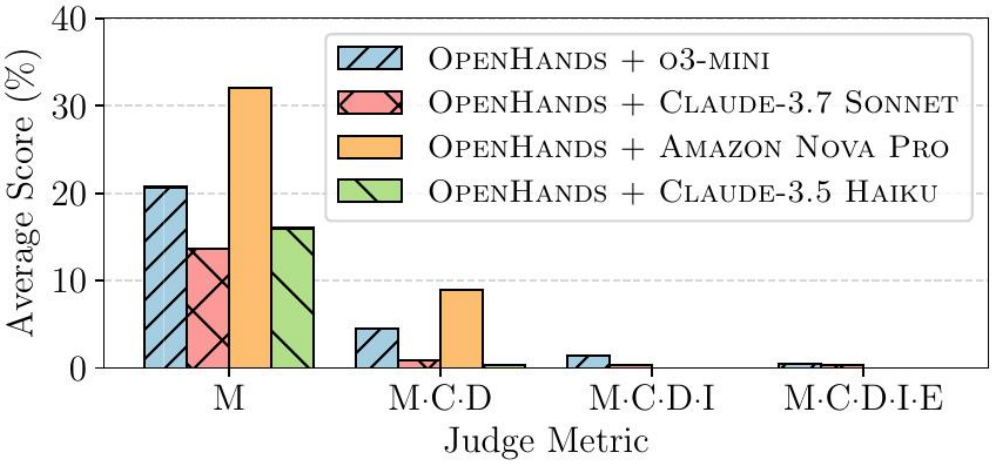

Figure 5: Stricter metrics (e.g., joint correctness) reveal lower true correctness compared to single-phase metrics.

Table 1: Average benchmark scores for various models when tested against various evaluation metrics. Popular Agents and LLMs perform poorly on EXP-Bench, showcasing its difficulty.

Agent	Model	D	I	E	I·E	C	All✓	All·E✓	#E
OpenHands	o3-mini	18.4	20.3	15.0	2.9	21.0	1.4	0.5	420
OpenHands	Claude-3.7 Sonnet	16.0	35.0	33.2	14.9	13.4	0.7	0.4	235
OpenHands	Amazon Nova Pro	18.2	19.5	26.8	0.0	15.7	0.0	0.0	56
OpenHands	Claude-3.5 Haiku	20.6	26.2	9.3	1.3	13.8	0.0	0.0	237
OpenHands	DeepSeek R1	6.8	10.0	0.7	0.0	2.4	0.0	0.0	140
IterativeAgent	Claude-3.5 Haiku	6.4	20.6	25.2	5.4	2.2	0.0	0.0	111
IterativeAgent	Amazon Nova Pro	0.1	10.0	18.1	0.0	0.3	0.0	0.0	215

6 Results

Table 1 reports average scores across metrics. Agents demonstrate partial correctness on individual phases but rarely satisfy all phases simultaneously. For example, OH+o3-mini achieves D=18.4%, I=20.3%, E=15.0%, C=21.0%, with All✓=1.4% and All•E✓=0.5% over 420 execution-checked

Table 2: Average benchmark scores of various models and agents across select task categories; see Supp. **??** for complete list. Evaluation performed against EXP-Bench.

Category	Agent	Model	D	I	E	I·E	C	All✓	All✓·E
Applications	OH	Nova Pro	19.2	23.9	19.0	0.0	13.9	0.0	0.0
Applications	OH	o3-mini	9.0	8.0	0.0	0.0	8.3	0.0	0.0
Applications	OH	3.5 Haiku	19.2	24.5	8.3	5.6	8.3	0.0	0.0
Applications	OH	3.7 Sonnet	9.0	26.8	30.8	7.7	8.3	2.8	0.0
Applications	IA	Nova Pro	0.0	9.8	5.0	0.0	0.0	0.0	0.0
Applications	IA	3.5 Haiku	0.0	18.0	0.0	0.0	0.0	0.0	0.0
Applications	OH	DeepSeek R1	3.1	4.3	0.0	0.0	0.0	0.0	0.0
RL	OH	3.7 Sonnet	18.3	48.2	27.3	21.2	17.6	2.0	3.0
RL	OH	o3-mini	23.5	34.8	15.7	2.0	27.5	3.9	0.0
RL	OH	3.5 Haiku	27.7	41.4	11.5	0.0	17.6	0.0	0.0
RL	IA	3.5 Haiku	3.3	27.5	17.4	0.0	2.4	0.0	0.0
RL	OH	DeepSeek R1	5.0	10.3	0.0	0.0	2.0	0.0	0.0
RL	IA	Nova Pro	0.0	8.6	9.7	0.0	0.0	0.0	0.0
RL	OH	Nova Pro	17.9	28.9	0.0	0.0	13.7	0.0	0.0

AI Generated

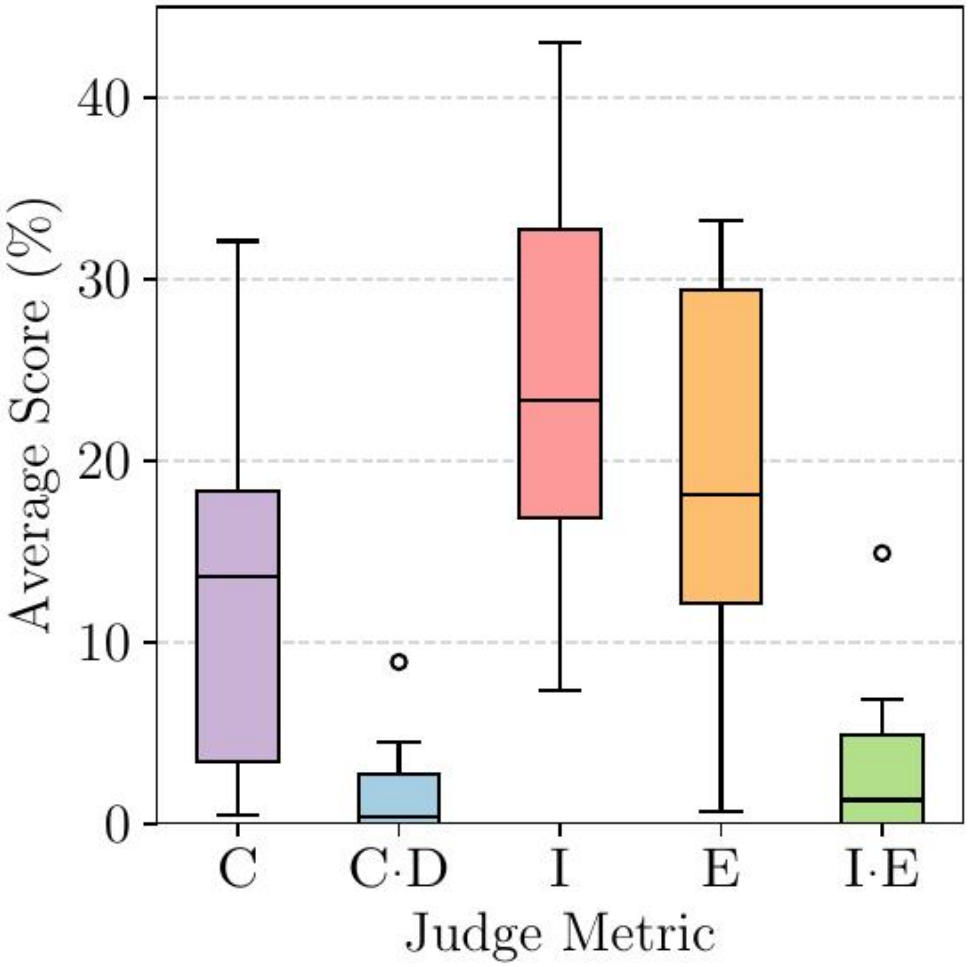

Figure 6: Stability analysis under scoring and sampling variations.

cases; OH+Claude-3.7 Sonnet attains higher I (35.0%) and E (33.2%) but All•E✓ remains 0.4% (n=235). Conjunctive scoring substantially reduces apparent performance relative to per-phase metrics.

Category-sliced results (Table 2) indicate that RL tasks are relatively stronger in implementation and I•E, e.g., OH+Sonnet I=48.2% and I•E=21.2%, yet overall end-to-end completion is still low.

Figure 7 illustrates cost–time trade-offs across agents, while Figure 5 shows that stricter metrics reveal lower true correctness. Together, these figures highlight sensitivity to evaluation choices and resource budgets.

7 Analysis

We analyze error modes and diagnostic slices to assess what EXP-Bench reveals beyond holistic scores. The failure taxonomy in Table 3 shows prevalent issues: missing essential implementation components (39.7%), environment/dependency errors (29.4%), script/file errors (23.8%), misclassified design variables (16%), and missing or incorrect conclusions (26.2%/19.7%). These patterns indicate that agents often falter at handoffs between phases (e.g., design-to-implementation) and during environment setup.

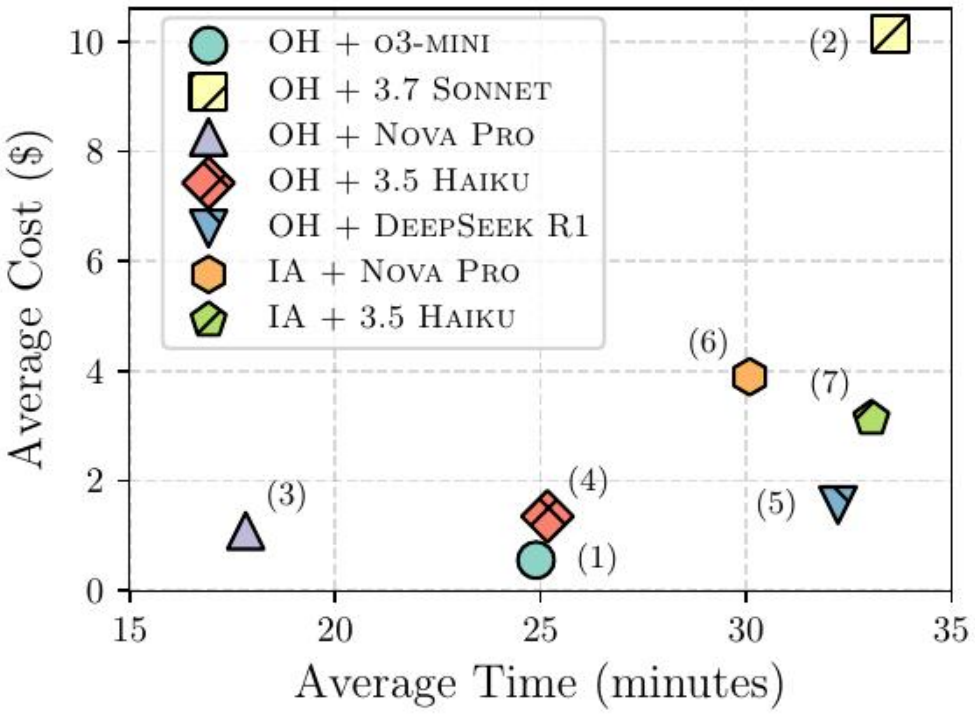

Figure 7: Cost–time trade-offs and performance variation across agents.

Conjunctive checks serve as a diagnostic lens: many tasks register progress on D or I but fail at E or C, demonstrating brittle pipelines. We further observe that some categories (e.g., RL) show higher I and I•E but still low end-to-end completion, suggesting that producing runnable code does not guarantee successful execution under constraints or correct interpretation.

Beyond aggregate scores, we examine metric sensitivity and adjudication reliability. Using the 200-item calibration set with human reviewers, judge–human agreement is highest for Execution (objective signals) and lowest for Conclusion (logical inference from noisy runs). Most disagreements arise from ambiguous success criteria and partial-credit cases; tightening rubrics and requiring joint phase consistency reduces variance without flipping many task-level decisions. Across repeated resamplings of tasks and seeds, rankings remain qualitatively stable for D/I/E/C but become less stable for All✓ due to many near-zero scores; relative gaps among the strongest systems persist. Finally, we apply basic contamination checks (string matching and repo masking) and hash-locked containers to reduce trivial memorization. These checks support the benchmark's diagnostic value while highlighting Conclusion scoring as a primary reliability bottleneck.

8 Discussion

What EXP-Bench measures. EXP-Bench measures the ability of agents to complete end-to-end research experiments under realistic constraints, requiring alignment between design, implementation, execution, and conclusions.

What EXP-Bench does not measure. EXP-Bench does not claim to assess all aspects of scientific discovery, long-horizon project management, or creativity; it focuses on reproducible execution of specified experiments derived from papers with public code.

Intended use. We envision EXP-Bench as a diagnostic suite for agentic systems, platform interfaces, and toolchains (e.g., IDEs, sandboxes), complementing broader benchmarks [2, 3] and agent evaluations [6, 7, 1].

Table 3: Agents fail in diverse ways across different phases of experimentation; this table presents a simplified subset of common examples, measured across all agent and model evaluations.

Phase	Failure Type	Prevalence (%)
Design	Incomplete or Misclassified Design Variables	16.05
Design	Irrelevant Procedural Additions in Design	7.62
Implementation	Missing Essential implementation Components	39.71
Implementation	Incomplete Evaluation Metric Implementation	2.15
Implementation	Incomplete Data and Preprocessing Setup	1.83
Execution	Environment/Dependency Configuration Errors	29.38
Execution	Execution Script and File Errors	23.84
Execution	Missing Setup Script File	6.95
Execution	Tensor Operation Execution Error	3.22
Conclusion	Missing Conclusion Content	26.18
Conclusion	Incorrect Conclusion Interpretation	19.66
Conclusion	Extraneous Details in Conclusion	7.77
Conclusion	Incorrect Numeric Conclusion	3.21

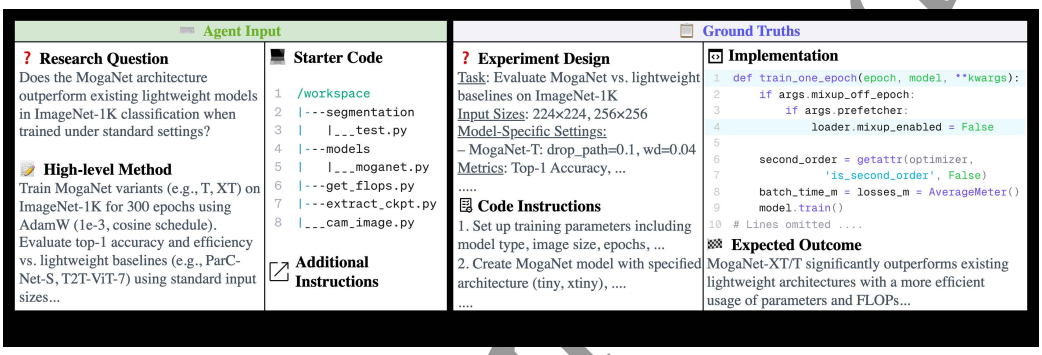

Figure 8: Example EXP-Bench task. Agents receive a natural-language research goal, a masked repository with entry points, and constraints (budgets, timeouts). Success requires coherent design, code edits, execution within the container, and a justified conclusion mapped to ground-truth outcomes. This illustrates the end-to-end nature of EXP-Bench beyond single-phase tests.

Reporting recommendations. To support comparability, we release a reporting template that records hardware, container image digests, timeouts, retries, seeds, and per-task transcripts. We recommend publishing both phase-wise and conjunctive metrics, stratified by category and difficulty, and including cost–time curves. To reduce judge variance, we provide a 200-item calibration set with human adjudication and ask authors to submit raw logs for spot checks.

Limitations. Scope is currently limited to NeurIPS/ICLR 2024 papers with public repositories, which may bias domains and tooling; extraction relies on automated parsing and LLM judgments (for D/I/C), introducing potential annotation noise and rubric sensitivity; execution constraints (hardware, timeouts) can disadvantage tasks with heavier training or preprocessing demands; despite masking and monitoring, contamination and leakage remain risks; and evaluation fidelity depends on both environment reproducibility and the quality of ground-truth scripts. We surface these caveats to prevent over-interpretation of scores.

9 Conclusion

We present EXP-Bench, a benchmark for evaluating AI agents on executable research experiments grounded in peer-reviewed papers and public code. EXP-Bench combines semi-automated task construction with execution-based validation and a conjunctive evaluation protocol across design, implementation, execution, and conclusion. Experiments across multiple agents and models show partial phase-wise competence but extremely low end-to-end completion when joint correctness is required. We hope EXP-Bench will encourage more rigorous, reproducible evaluation of agentic systems and catalyze progress on tools and interfaces that support complete scientific workflows.

AI-Generated

References

[1] Z. Chen, S. Chen, Y. Ning, Q. Zhang, B. Wang, B. Yu, Y. Li, Z. Liao, C. Wei, Z. Lu, et al. Scienceagentbench: Toward rigorous assessment of language agents for data-driven scientific discovery. *arXiv preprint arXiv:2410.05080*, 2024.

[2] Google. BIG-bench. 2022. `https://github.com/google/BIG-bench`.

[3] D. Hendrycks, C. Burns, S. Basart, A. Zou, M. Mazeika, D. Song, and J. Steinhardt. Measuring massive multitask language understanding, 2021.

[4] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan. Swe-bench: Can language models resolve real-world github issues? *arXiv preprint arXiv:2310.06770*, 2023.

[5] S. Li, Z. Wang, Z. Liu, C. Tan, H. Lin, D. Wu, Z. Chen, J. Zheng, and S. Z. Li. Moganet: Multi-order gated aggregation network, 2024.

[6] X. Liu, H. Yu, H. Zhang, Y. Xu, X. Lei, H. Lai, Y. Gu, H. Ding, K. Men, K. Yang, S. Zhang, X. Deng, A. Zeng, Z. Du, C. Zhang, S. Shen, T. Zhang, Y. Su, H. Sun, M. Huang, Y. Dong, and J. Tang. Agentbench: Evaluating llms as agents, 2023.

[7] X. Wang, B. Li, Y. Song, F. F. Xu, X. Tang, M. Zhuge, J. Pan, Y. Song, B. Li, J. Singh, et al. Openhands: An open platform for ai software developers as generalist agents. *arXiv preprint arXiv:2407.16741*, 2024.

AI-Generated